

Machine Learning

Lecture 3: Aggregation



國立臺灣大學
National Taiwan University



National Taiwan University

Department of Computer Science and Information Engineering

Prof. Chun-Yi Lee

Outline

- Recap: Optimization
- Aggregation



Recap: Optimization

Quasi-Newton's Method - Without Computing Full Hessian

- **Newton's method update rule**:
  - $\theta_{t+1} = \theta_t - H_{\mathscr{L}}(\theta_t)^{-1} \nabla \mathscr{L}(\theta_t)$
- **Quasi-Newton's method update rule**:
  - $\theta_{t+1} = \theta_t - \alpha_t B_t^{-1} \nabla \mathscr{L}(\theta_t)$
  - Approximate the Hessian matrix $H_{\mathscr{L}}$ with a positive-definite matrix B
  - α_t is a **line search** parameter
- **Characteristics**:
  - Avoid explicit Hessian matrix computation
  - Uses gradient history for efficiency
  - Achieves near Newton's convergence with lower cost

$$H_{\mathscr{L}} = \begin{bmatrix} \frac{\partial^2 \mathscr{L}}{\partial (\theta^0)^2} & \frac{\partial^2 \mathscr{L}}{\partial \theta^0 \partial \theta^1} & \cdots & \frac{\partial^2 \mathscr{L}}{\partial \theta^0 \partial \theta^d} \\ \frac{\partial^2 \mathscr{L}}{\partial \theta^1 \partial \theta^0} & \frac{\partial^2 \mathscr{L}}{\partial (\theta^1)^2} & \cdots & \frac{\partial^2 \mathscr{L}}{\partial \theta^1 \partial \theta^d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 \mathscr{L}}{\partial \theta^d \partial \theta^0} & \frac{\partial^2 \mathscr{L}}{\partial \theta^d \partial \theta^1} & \cdots & \frac{\partial^2 \mathscr{L}}{\partial (\theta^d)^2} \end{bmatrix}$$

- $H_{\mathscr{L}}$
  - Computationally expensive
  - Ill-conditioned problem
  - Not guaranteed to be positive-definite

Recap: Optimization

Quasi-Newton Method - Broyden-Fletcher-Goldfarb-Shanno (BFGS) Algorithm

1. **Initialization**: choose an initial guess θ_0
   - Set $B_0 = I$ (identity matrix as initial Hessian approximation)
2. **Search direction**: $p_t = -B_t^{-1} \nabla \mathscr{L}(\theta_t)$
3. **Line search**
   - Use backtracking line search to find the step size α_t
   - Update: $\theta_{t+1} = \theta_t + \alpha_t p_t$
4. **Stopping criterion**
   - If convergence is reached, stop the algorithm
5. **Gradient difference**: $s_t = \theta_{t+1} - \theta_t$, $y_t = \nabla \mathscr{L}(\theta_{t+1}) - \nabla \mathscr{L}(\theta_t)$
6. **Update inversion Hessian approximation**

$$B_{t+1}^{-1} = \left(I - \frac{s_t y_t^T}{y_t^T s_t} \right) B_t^{-1} \left(I - \frac{y_t s_t^T}{y_t^T s_t} \right) + \frac{s_t s_t^T}{y_t^T s_t}$$

- Advantage:
  - Near-Newton speed
- Limitation:
  - Storage cost: $O(d^2)$

We are building up **curvature information** gradually by observing how gradients change as we move through parameter space.

Recap: Optimization

Quasi-Newton Method - Broyden-Fletcher-Goldfarb-Shanno (BFGS) Algorithm

Example:

$$\mathscr{L}(\theta) = \frac{1}{2}\theta^T H\theta - b^T\theta \text{, where } H = \begin{bmatrix} 4 & 1 & 0.5 \\ 1 & 3 & 0.2 \\ 0.5 & 0.2 & 2 \end{bmatrix}, b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

- Newton method: $\theta^* = H^{-1}b \approx \begin{bmatrix} 0.2799 & -0.0892 & -0.0611 \\ -0.0892 & 0.364 & -0.0141 \\ -0.0611 & -0.0141 & 0.5167 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \approx \begin{bmatrix} -0.081 \\ 0.597 \\ 1.461 \end{bmatrix}$
- BFGS algorithm: (The α_t is set to 1 for simplicity)

Iteration 1

1. Set $\theta_0 = [0, 0, 0]^T, B_0^{-1} = I$
2. $p_0 = -B_0^{-1}\nabla\mathscr{L}(\theta_0) = b = [1, 2, 3]^T$
3. $\theta_1 = \theta_0 + \alpha_0 p_0 = [1, 2, 3]^T$
4. $s_0 = \theta_1 - \theta_0 = [1, 2, 3]^T, \quad y_0 = \nabla\mathscr{L}(\theta_1) - \nabla\mathscr{L}(\theta_0) = H\theta_1 = [7.5, 7.6, 6.9]^T$
5. $B_1^{-1} = \left(I - \frac{s_0 y_0^T}{y_0^T s_0}\right) B_0^{-1} \left(I - \frac{y_0 s_0^T}{y_0^T s_0}\right) + \frac{s_0 s_0^T}{y_0^T s_0} \approx \begin{bmatrix} 0.763 & -0.303 & -0.351 \\ -0.303 & 0.735 & -0.190 \\ -0.351 & -0.190 & 1.026 \end{bmatrix}$



Recap: Optimization

Quasi-Newton Method - Broyden-Fletcher-Goldfarb-Shanno (BFGS) Algorithm

Iteration 2

1. $\theta_1 = [1, 2, 3]^T, B_1^{-1} = \begin{bmatrix} 0.763 & -0.303 & -0.351 \\ -0.303 & 0.735 & -0.190 \\ -0.351 & -0.190 & 1.026 \end{bmatrix}$

2. $p_1 = -B_1^{-1}\nabla\mathcal{L}(\theta_1) = b = [-1.895, -1.404, -0.654]^T$

3. $\theta_2 = \theta_1 + \alpha_1 p_1 = [-0.895, 0.596, 2.346]^T$

4. $s_1 = \theta_2 - \theta_1 = [-1.895, -1.404, -0.654]^T, \quad y_1 = \nabla\mathcal{L}(\theta_2) - \nabla\mathcal{L}(\theta_1) \approx [-9.312, -6.238, -2.537]^T$

5. $B_2^{-1} = \left(I - \frac{s_1 y_1^T}{y_1^T s_1}\right) B_1^{-1} \left(I - \frac{y_1 s_1^T}{y_1^T s_1}\right) + \frac{s_1 s_1^T}{y_1^T s_1} \approx \begin{bmatrix} 0.506 & -0.364 & -0.214 \\ -0.364 & 0.786 & -0.044 \\ -0.214 & -0.044 & 1.151 \end{bmatrix}$



Iteration 7

$\theta_7 \approx [-0.082, 0.597, 1.461]^T, \quad B_7^{-1} \approx \begin{bmatrix} 0.2811 & -0.0900 & -0.0625 \\ -0.0900 & 0.365 & -0.0140 \\ -0.0625 & -0.0140 & 0.5196 \end{bmatrix} \quad H^{-1} = \begin{bmatrix} 0.2799 & -0.0892 & -0.0611 \\ -0.0892 & 0.364 & -0.0141 \\ -0.0611 & -0.0141 & 0.5167 \end{bmatrix}$

Recap: Optimization

Quasi-Newton Method - Limited-memory BFGS (L-BFGS) Algorithm

Efficiently approximate search direction without storing full Hessian B_t^{-1}

L-BFGS Algorithm

m is the length of saving s_t and y_t
for $t = 1, \ldots, T$ **do**
  $q = \nabla \mathcal{L}(\theta_t)$
  for $i = t-1, t-2, \ldots, t-m$ **do** ▷ backward loop
    $\alpha_i = \rho_i s_i^T q$
    $q = q - \alpha_i y_i$
  end for
  $\gamma = \frac{s_{t-1}^T y_{t-1}}{y_{t-1}^T y_{t-1}}$ (if $t > 0$, else $\gamma = 1$)
  $r = \gamma q$
  for $i = t-m, t-m+1, \ldots, t-1$ **do** ▷ forward loop
    $\beta_i = \rho_i y_i^T r$
    $r = r + s_i(\alpha_i - \beta_i)$
  end for
  $p_t = -r$
  $\theta_{t+1} = \theta_t + \alpha_t p_t$
end for

Advantage

- Low memory use: store only a few vectors ($O(md)$), instead of the full B^{-1} ($O(d^2)$)
- Fast convergence (compared to GD)
- Scale to large problems

Recap: Optimization

Conjugate Gradient Descent

Goal:

- Accelerate the convergence rate of steepest descent
- Avoid the high computational cost of Newton's method

The conjugate gradient descent is originally developed for solving **quadratic problem**:

$$\min_{\theta \in \mathbb{R}^n} \frac{1}{2}\theta^T A\theta - b^T\theta, \text{ assume matrix } A \in \mathbb{R}^{nxn} \text{ is positive definite}$$

$$\mathscr{L}(\theta) = \frac{1}{2}\theta^T A\theta - b^T\theta \Rightarrow \nabla\mathscr{L}(\theta) = A\theta - b \Rightarrow \nabla\mathscr{L}(\theta) = 0,\ A\theta = b$$

Solve $A\theta = b$ is equivalent to minimize the quadratic objective

Assumptions:

- The matrix $A \in \mathbb{R}^{nxn}$ is **symmetry positive definite**
- A set of vector $\{u_1, u_2, \ldots, u_K | u_i \in \mathbb{R}^n\}$ are A-orthogonal (conjugate) with respect to A, if $u_i^T A u_j = 0, \forall i \neq j$

Recap: Optimization

Conjugate Gradient Descent Algorithm

Conjugate Gradient Descent Algorithm

Require: Symmetric positive definite matrix A, vector b, initial guess θ_0

$r_0 \leftarrow b - A\theta_0$ ▷ Initial residual

$p_0 \leftarrow r_0$ ▷ Initial search direction

for $t = 0, 1, 2, \ldots$ until convergence **do**

  $\alpha_t \leftarrow \frac{r_t^T r_t}{p_t^T A p_t}$

  $\theta_{t+1} \leftarrow \theta_t + \alpha_t p_t$

  $r_{t+1} \leftarrow r_t - \alpha_t A p_t$

  if $\|r_{t+1}\| < \epsilon$ **then**

    break

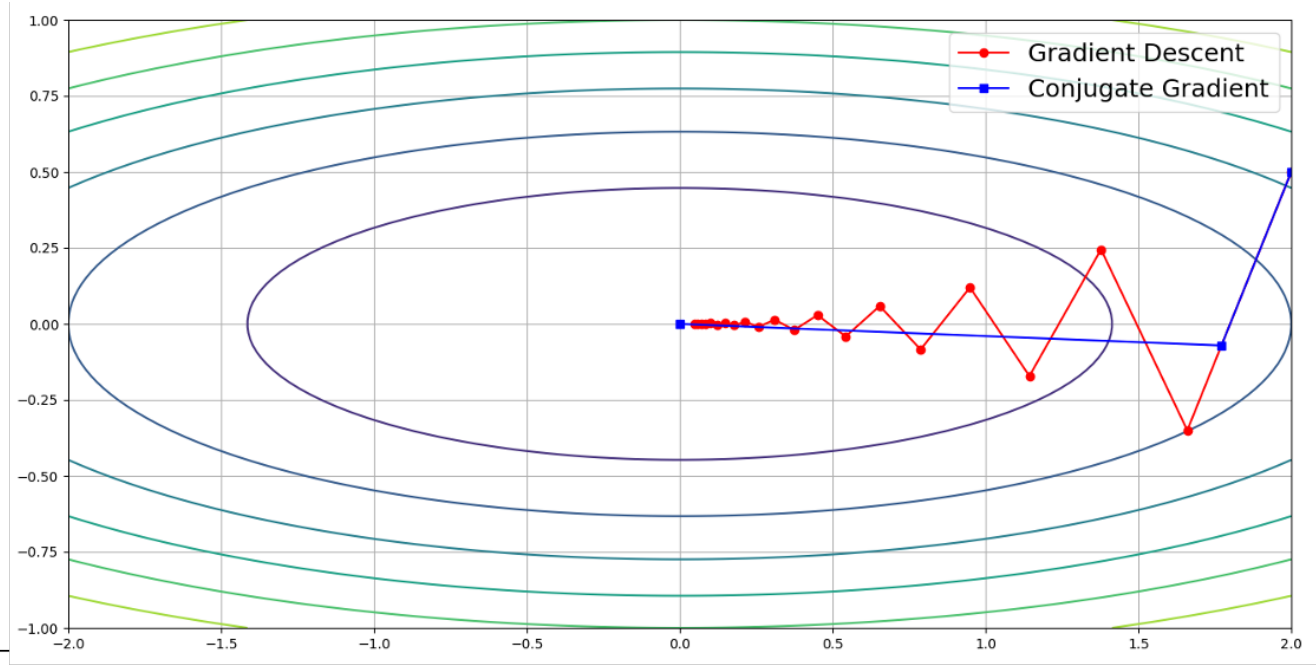
  end if

  $\beta_t \leftarrow \frac{r_{t+1}^T r_{t+1}}{r_t^T r_t}$

  $p_{t+1} \leftarrow r_{t+1} + \beta_t p_t$

end for

return θ_{t+1}



Reference: https://www.cs.cmu.edu/~pradeepr/convexopt/Lecture_Slides/conjugate_direction_methods.pdf

Recap: Optimization

Conjugate Gradient Descent

Intuition:

- Instead of **just following the gradient**, it builds a new search direction each time that is independent of the previous ones in a special sense
- This guarantees that every step makes progress without **undoing** earlier work



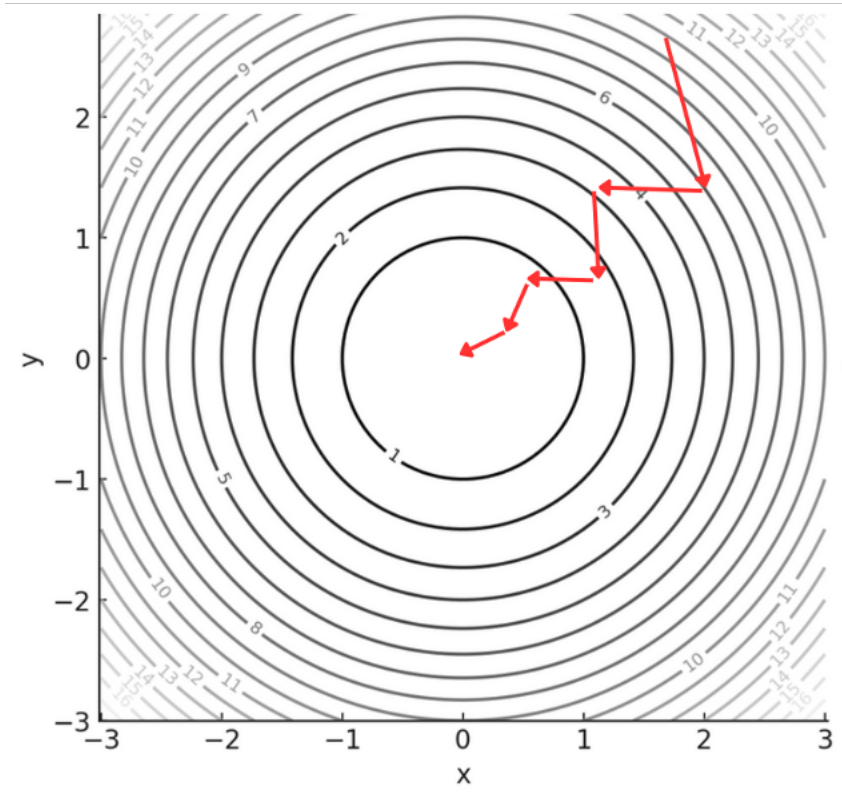
Advantage

- Finite convergence: solving in at most d steps, where d is the dimension of x (proof in the reference)
- Does not require the Hessian matrix
- Efficiency: faster than gradient descent
- Stability: more numerically stable than Newton's method

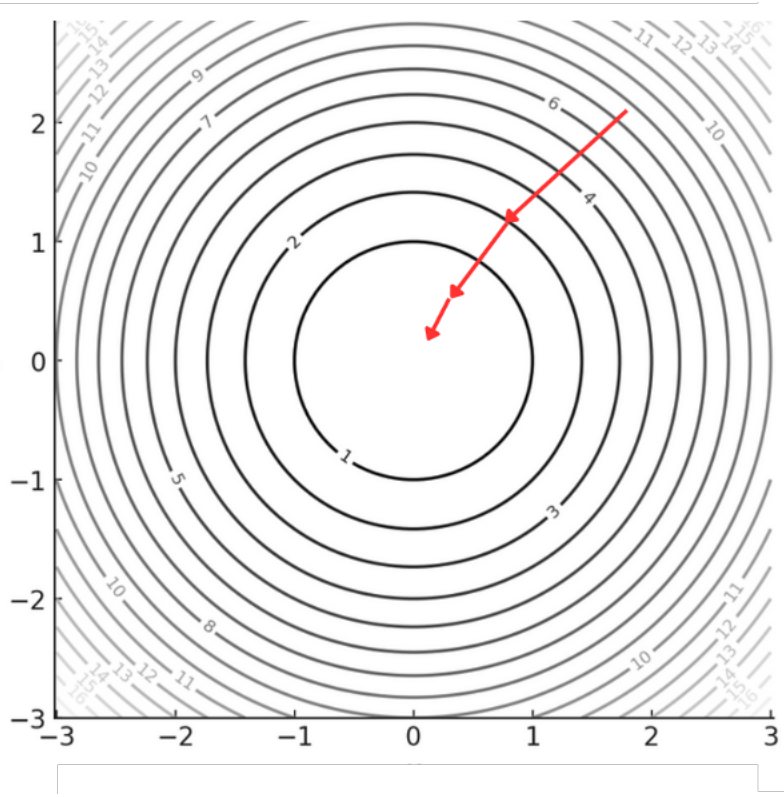
Reference: https://www.cs.cmu.edu/~pradeepr/convexopt/Lecture_Slides/conjugate_direction_methods.pdf

Recap: Optimization

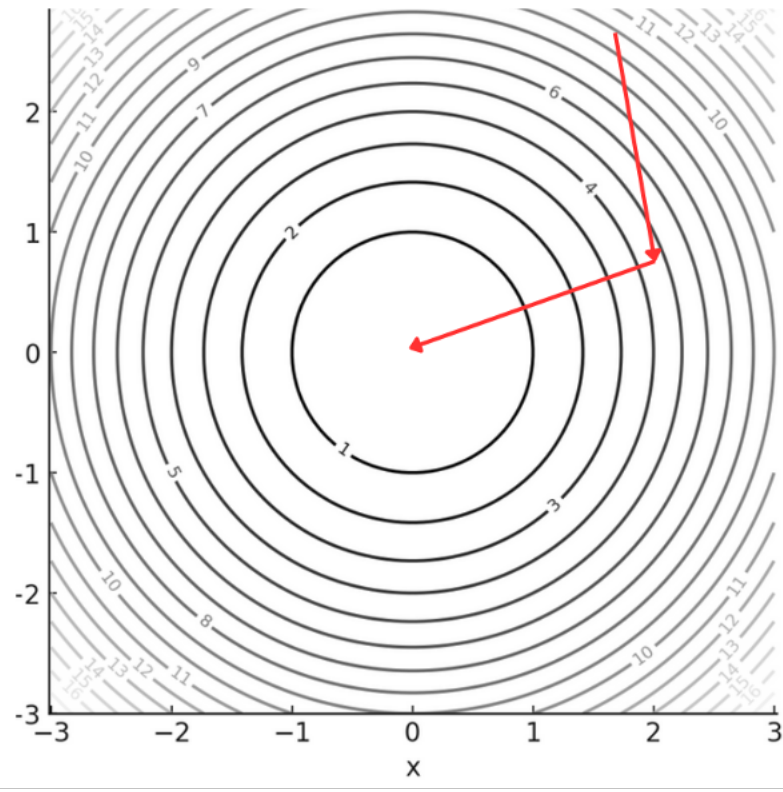
Comparison of Different Optimization Methods



Gradient Descent



Newton's Method



Conjugate Gradient Descent

Outline

- Recap: Optimization
- **Aggregation**



Aggregation

Introduction

Combine multiple models (often called base-learners) to produce better prediction

- This inductive bias leads to error if the assumptions do not hold for the data
- Learning is an ill-posed problem and with finite data, each algorithm converges to a different solution and fails under different circumstances
- By suitably combining multiple **base-learners**, accuracy can be improved

There are two questions here

1. How do we generate base-learners that complement each other?
2. How do we combine the outputs of base-learners for maximum accuracy?

Aggregation

Generate Diverse Learners

- **Different algorithms**:
  - Use different learning algorithms introduces varied assumptions about the data, which produce diverse classifiers
- **Different hyperparameters**:
  - Apply the same algorithm with different hyperparameters settings can lead to different models
- **Different input representation**:
  - Transform or encode the input differently can highlight distinct features
  - Different learners will look at the same problem from different points of view
- **Different training sets**:
  - Train base-learners on different subsets of the data
  - Example: Bagging, Boosting

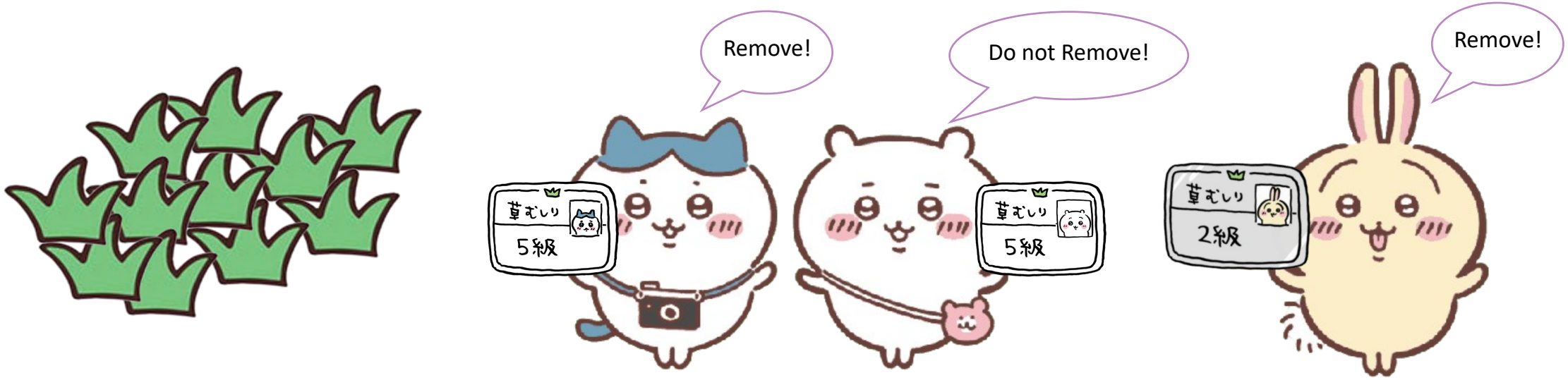
Aggregation

Combine Predictions from Multiple Models to Produce a Final Output

- Assume we have L base-learners, $d_l(x)$ denotes the prediction of l-th base-learner, and the final prediction combined by base-learners can be represented as:

$$\hat{y} = g_\phi(d_1, d_2, \ldots, d_L),\ g(\cdot) \text{ is the combining function, and } \phi \text{ denotes its parameters}$$

- Each learner can predict N outputs $d_l(x_i)$, $l = 1,...,L$, $i = 1,...,N$. By combining them, we can yield N predictions for every sample in the dataset as $\hat{y}_i$, $i = 1,...,N$.



Aggregation

Different Combination Rules

- In **classification**, the simplest aggregation method (or ensemble strategy) is **voting**, which corresponds to taking a linear combination of the learners.

Selection: $\hat{y} = d_{l*}(x), l* = \underset{l \in \{1,2,\ldots,L\}}{\operatorname{argmin}} \; Err(d_l)$

Non-uniform: $\hat{y} = \operatorname{sign}\left(\sum_{t=1}^{T} \phi_l \cdot d_l(x) \right)$, where $\phi_l \geq 0$

Uniform: $\hat{y} = \operatorname{sign}\left(\sum_{t=1}^{T} 1 \cdot d_l(x) \right)$

Condition: $\hat{y} = \operatorname{sign}\left(\sum_{t=1}^{T} q_l(x) \cdot d_l(x) \right)$, where $q_l(x) \geq 0$

- In **regression**, the outputs of base regressors can be aggregated by simple mean, weighted mean, or median. The median is generally more robust to noise than the mean. The fusion functions are listed:

Uniform: $\hat{y}_i = \sum_{l=1}^{L} \phi_l \cdot d_l(x_i)$, where $\phi_l = \frac{1}{L}$

Minimum: $\hat{y}_i = \min_l d_l(x_i)$

Non-uniform: $\hat{y}_i = \sum_{l=1}^{L} \phi_l \cdot d_l(x_i)$, where $\phi_l \geq 0$, $\sum_{l=1}^{L} \phi_l = 1$

Maximum: $\hat{y}_i = \max_l d_l(x_i)$

Median: $\hat{y}_i = \operatorname{median}_l d_l(x_i)$

Product: $\hat{y}_i = \prod_{l=1}^{L} d_l(x_i)$

Aggregation

Bootstrap Aggregation (Bagging)

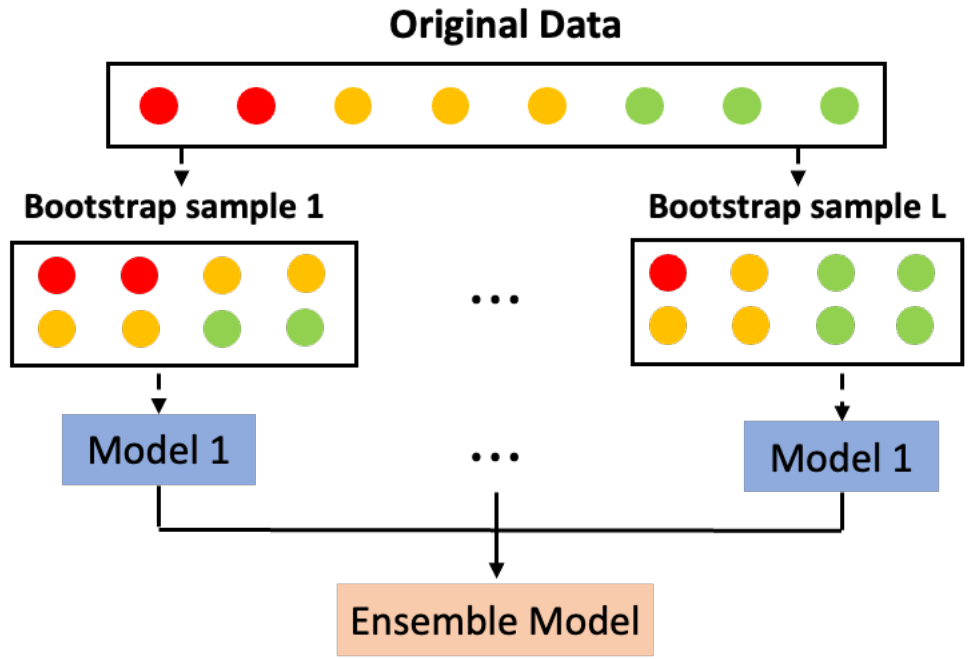
- **Aggregation**: Combine multiple predictors to make a single stronger predictor
- **Bootstrap**: Each bootstrap sample is slightly different from the original data because some points may be duplicated and some may be left out
- **Bagging**: Ensemble method that trains multiple models on bootstrap-resampled versions of the training data, then combines their outputs.

Bootstrapping

(Sampling with replacement)

Model Training

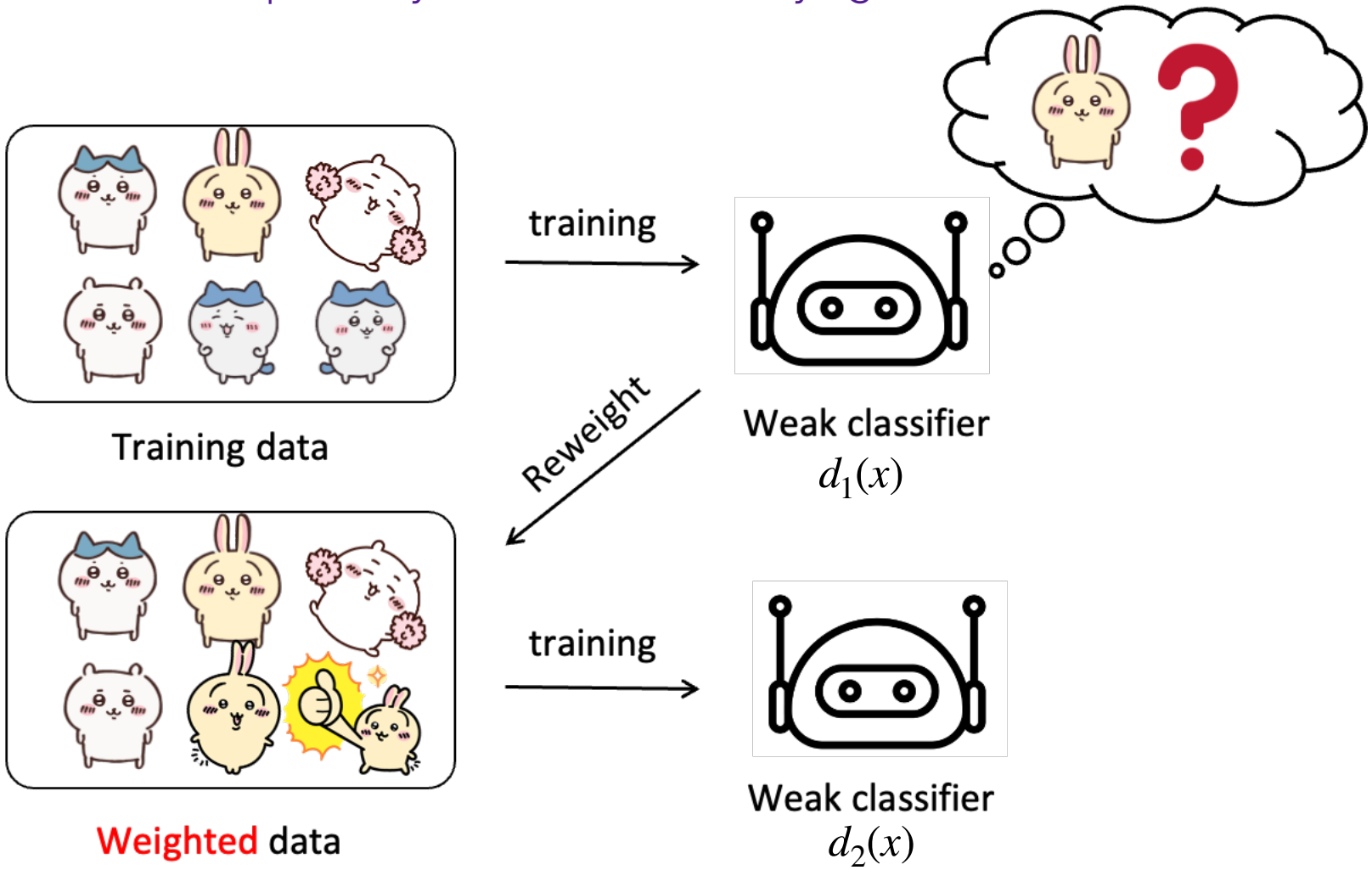
(Linear model, SVM, Logistic regression)



Aggregation

Boosting

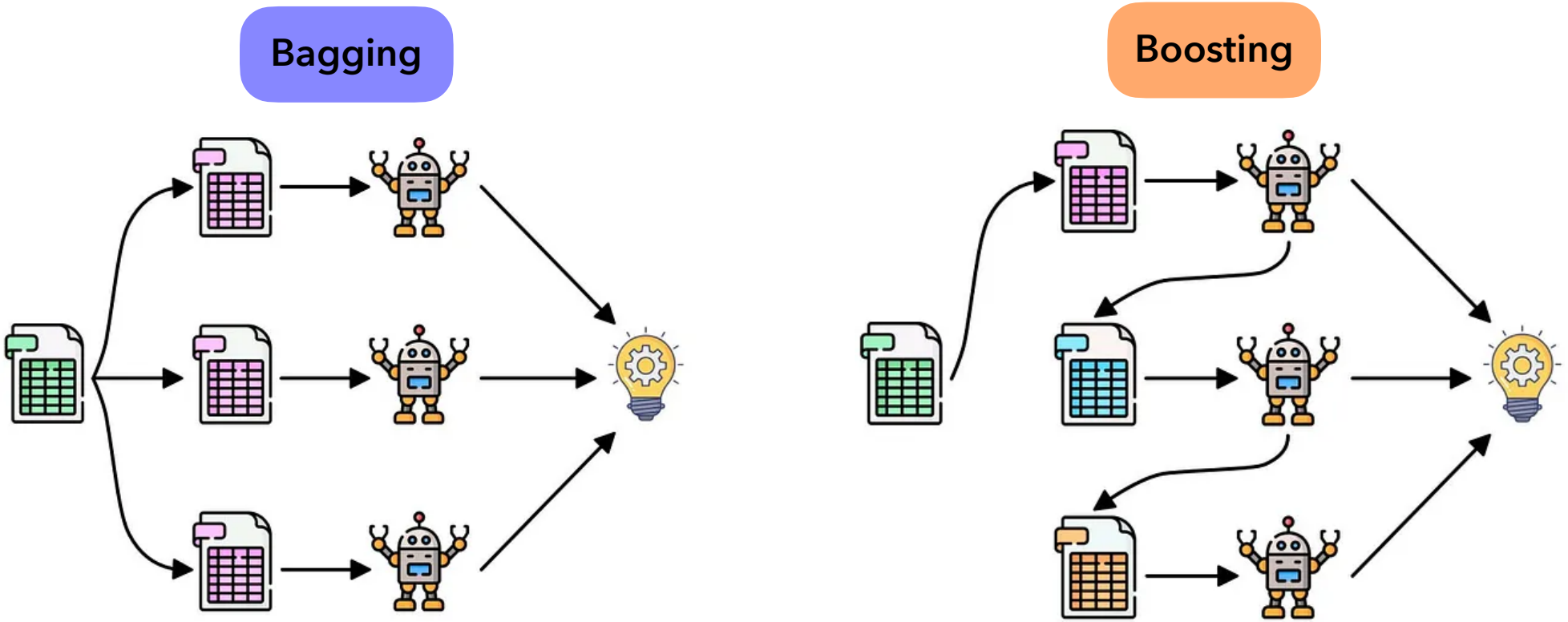
Boosting combines multiple models sequentially, with each model trying to correct the mistakes of the previous ones.



Aggregation

Boosting

- In **bagging**, diversity of base-learners is introduced by chance
- In **boosting**, diversity is introduced actively by focusing on previous errors

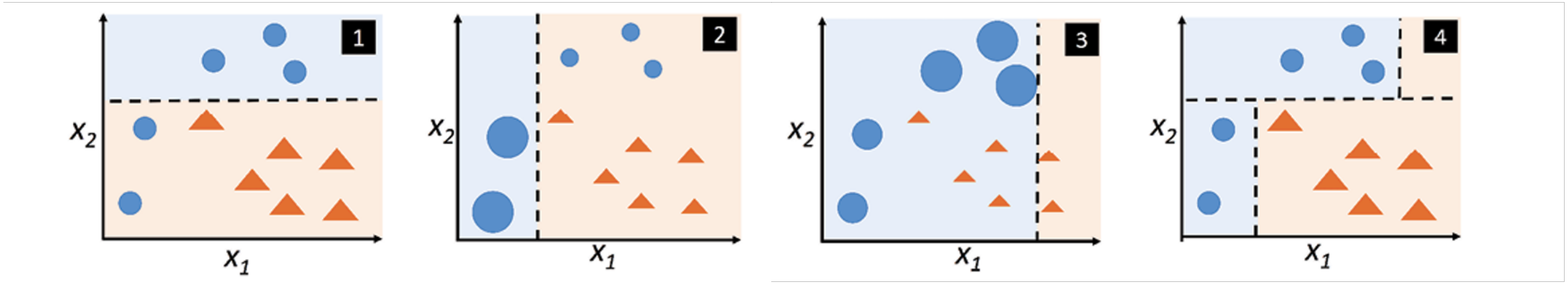


Reference: https://medium.com/@roshmitadey/bagging-v-s-boosting-be765c970fd1

Aggregation

Adaptive Boosting (AdaBoost)

- Train weak classifiers sequentially, with each new classifier giving more weight to instances misclassified by previous classifier.
- AdaBoost can focus on difficult instances to improve the overall accuracy
- **Combine** weak classifiers based on its accuracy to make a final prediction



Reference: https://vitalflux.com/adaboost-algorithm-explained-with-python-example/

Aggregation

Adaptive Boosting (AdaBoost) Algorithm

Adaboost Algorithm (Simplified Version)

1: **Train a weak classifier** on training data with current sample weights.
2: **Update the weights:**
   – Increase the weight of misclassified instances.
   – Decrease the weight of correctly classified instances.
3: **Train the next weak classifier** using the updated weights.
4: Repeat steps 2 and 3 for L times to generate L classifiers.
5: **Combine all weak classifiers**, weighted by their error rates, into a strong classifier.

Aggregation

Adaptive Boosting (AdaBoost) Algorithm

Adaboost Algorithm

1: Initial training data $\{(x_n, y_n, u_1^n)\}_{n=1}^N$, where $y_n \in \{-1, 1\}$, and $u_1^n = 1$ for all $n = 1, \ldots, N$.

2: Train a weak classifier $d_1(x)$ with $\{(x_n, y_n, u_1^n)\}_{n=1}^N$ such that the error rate $\varepsilon_1 < 0.5$:

$$\varepsilon_1 = \frac{\sum_{n=1}^N u_1^n \, \delta(d_1(x_n) \neq y^n)}{\sum_{n=1}^N u_1^n} < 0.5,$$

where

$$\delta(\text{condition}) = \begin{cases} 1, & \text{if the condition is true,} \\ 0, & \text{if the condition is false.} \end{cases}$$

3: Update the weights $u_2 \leftarrow u_1$:

- If x_n misclassified by d_1, $d_1(x_n) \neq y_n$, then $u_2^n \leftarrow u_1^n \times r_1$ (increase weight)
- If x_n correctly classified by d_1, $d_1(x_n) = y_n$), then $u_2^n \leftarrow u_1^n / r_1$ (decrease weight)

such that

$$\frac{\sum_{n=1}^N u_2^n \, \delta(d_1(x_n) \neq y_n)}{\sum_{n=1}^N u_2^n} = 0.5$$

4: With the updated weights, the performance of d_1 would be no better than random guessing.

How to determine r_l to change the weight for each data?

AdaBoost adjusts weights so the previous classifier's weighted error becomes 0.5

Aggregation

Adaptive Boosting (AdaBoost) Algorithm

Adaboost Algorithm (cont.)

1: Train $d_2(x)$ with the new weighted data $\{(x_n, y_n, u_2^n)\}_{n=1}^N$ to get a better classifier such that $\varepsilon_2 < 0.5$.
2: Repeat step 2, 3 to get $d_2(x), d_3(x), \ldots, d_T(x)$.
3: Boost weak classifiers iteratively to obtain a strong classifier:

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \phi_t d_t(x)\right),$$

where

$$\phi_t = \ln\sqrt{\frac{1-\varepsilon_t}{\varepsilon_t}}, \qquad \text{sign}(x) = \begin{cases} 1, & x \geq 0, \\ -1, & x < 0. \end{cases}$$

Expect monotonic

$\varepsilon_t = \frac{1}{2} \implies \phi_t = 0$ (bad d_t, zero weight)

$\varepsilon_t = 0 \implies \phi_t = \infty$ (good d_t, superior weight)

Aggregation

Adaptive Boosting (AdaBoost) Algorithm

How to determine r_l to change the weight for each data?

Pre-knowledge:

$$\text{Define } Z_t = \sum_{n=1}^{N} u_t^n, \text{ since } \varepsilon_t = \frac{\sum_{n=1}^{N} u_t^n \, \delta(d_t(x_n) \neq y_n)}{Z_t},$$

$$\text{We have } \sum_{n=1}^{N} u_t^n \, \delta(d_t(x_n) \neq y_n) = Z_t \varepsilon_t, \quad \sum_{n=1}^{N} u_t^n \, \delta(d_t(x_n) = y_n) = Z_t(1-\varepsilon_t).$$

Determine r_l:

$$\frac{\sum_n u_{t+1}^n \, \delta(d_j(x_n) \neq y_n)}{\sum_n u_{t+1}^n} = 0.5$$

$$\text{Numerator: } \sum_{n=1}^{N} u_{t+1}^n \, \delta(d_t(x_n) \neq y_n) = \sum_{d_t(x_n) \neq y_n} u_t^n d_t$$

$$\text{Denominator: } \sum_{n=1}^{N} u_{t+1}^n = \sum_{d_t(x_n) \neq y_n} u_t^n d_t + \sum_{d_t(x_n) = y_n} \frac{u_t^n}{d_t}$$

$$\text{Then } \Rightarrow d_t \sum_{d_t(x_n) \neq y_n} u_t^n = \frac{1}{d_t} \sum_{d_t(x_n) = y_n} u_t^n \Rightarrow d_t Z_t \varepsilon_t = \frac{1}{d_t} Z_t(1-\varepsilon_t) \quad \Rightarrow \quad d_t = \sqrt{\frac{1-\varepsilon_t}{\varepsilon_t}}$$


Thank you for your attention!